Experiment 8

Student Name: Diwansh
Branch: CSE - AIML
Semester: 4
Subject Name: DBMS

UID: 24BAI70294
Section/Group: 24AIT_KRG G1
Date of Performance: 27/3/26
Subject Code: 24CSH-298

Aim
To understand the design and implementation of stored procedures in **PL/pgSQL (PostgreSQL)**, focusing on the use of parameter modes (**IN, OUT, INOUT**) to process employee data, encapsulate business logic, and return results efficiently.

Software Requirements

- **Database Management System:**
  - PostgreSQL
- **Client Tool:**
  - pgAdmin
  -

Objectives

- To create and execute a parameterized stored procedure that counts employees based on gender.
- To demonstrate the use of **IN, OUT, and INOUT parameters**.
- To execute the procedure using an **anonymous DO block**.
- To display results using RAISE NOTICE.
-

Practical / Experiment Steps

- **Table Creation:** Created an employees table with appropriate data types and primary key.
- **Data Insertion:** Inserted sample employee records with different genders and salaries.
- **Procedure Creation:** Developed a stored procedure get_employee_count_by_gender.
- **Parameter Usage:**

    - IN → accepts gender input
    - OUT → returns employee count
    - INOUT → tracks execution status
- **Query Logic:** Used SELECT COUNT(*) INTO to store result in output variable.
- **Anonymous Block:** Used a DO block to call the procedure and print results.
-

Procedure

- Connected to PostgreSQL database using pgAdmin.
- Created the employees table with columns: emp_id, emp_name, gender, and salary.
- Inserted sample records into the table.
- Defined the procedure with proper parameter modes (IN, OUT, INOUT).
- Implemented logic to count employees based on gender.
- Updated the STATUS variable to 'SUCCESS' inside the procedure.
- Declared variables in an anonymous block (GEN, COUNT_OF_EMPLOYEES, STATUS).
- Called the procedure using the CALL statement.
- Displayed results using RAISE NOTICE.
- Verified correctness by comparing with table data.

Input/Output Analysis

SQL Input Queries

```
CREATE TABLE EMPLOYEES(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(20),
gender VARCHAR(20),
salary NUMERIC(10, 2)
);
```

Output

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 92 msec.
```

SQL Input Queries

INSERT INTO employees (emp_id, emp_name, gender, salary) VALUES
(101, 'Amit', 'Male', 30000),
(102, 'Sarah', 'Female', 55000),
(103, 'Riya', 'Female', 45000),
(104, 'Arjun', 'Male', 59000),
(105, 'Anjali', 'Female', 66000);

Output

Data Output Messages Notifications

```
INSERT 0 5

Query returned successfully in 76 msec.
```

SQL Input Queries

CREATE OR REPLACE PROCEDURE get_employee_count_by_gender (

```
   IN IN_GENDER VARCHAR(20),
   OUT OUT_COUNT INT,
   INOUT STATUS VARCHAR(20)
)
AS
$$
BEGIN
   SELECT COUNT(*) INTO OUT_COUNT
   FROM employees
   WHERE gender = IN_GENDER;

   STATUS := 'SUCCESS';
END;
$$ LANGUAGE PLPGSQL;
```

Output

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 158 msec.
```

SQL Input Queries

```
DO
$$
DECLARE
GEN VARCHAR(20):='Female';
COUNT_OF_EMPLOYEES INT;
STATUS VARCHAR(20):='PENDING';

BEGIN
CALL get_employee_count_by_gender(GEN, COUNT_OF_EMPLOYEES,
STATUS);
```

```
RAISE NOTICE 'COUNT OF % EMPLOYEES IS % AND STATUS IS %',
GEN, COUNT_OF_EMPLOYEES, STATUS;
END;
$$;
```

Output

Data Output Messages Notifications

```
NOTICE:  COUNT OF Female EMPLOYEES IS 3 AND STATUS IS SUCCESS
DO

Query returned successfully in 46 msec.
```

Learning Outcomes

- Understood how to encapsulate SQL logic into reusable procedures to improve code maintainability.
- Gained expertise in using IN, OUT, and INOUT modes to pass data into and retrieve results from stored subprograms.
- Learned to bridge SQL aggregate functions with PL/SQL variables using the INTO keyword.
- Developed skills in using status flags and notice alerts to track the success and results of procedural calls.